

```

ment, fragment_size);
•
•      glShaderBinary(1, &vertShader, formats[i],
vertex, vertex_size);
•      compiled = GL_TRUE;
•      break;
•    }
•    delete [] formats;
•  }

```

The second approach to load the shader objects is to use the GLES2 Shader Compiler to read and compile the shader source code for both the fragment and the vertex shader.

```

•   else if(GL_TRUE == shaderCompiler)
•   {
•       // Read and compile the shader source code for
fragment shader
•       glShaderSource(fragShader, 1, (const
char*)&fragmentText, null);
•       glCompileShader(fragShader);
•       GLint bShaderCompiled = GL_FALSE;
•       // Determine whether the compilation is successful
•       glGetShaderiv(fragShader, GL_COMPILE_STATUS,
&bShaderCompiled);
•
•       // Read and compile the shader source code for
vertex shader
•       glShaderSource(vertShader, 1, (const
char*)&vertexText, null);
•       glCompileShader(vertShader);
•       // Determine whether the compilation is successful
•       glGetShaderiv(vertShader, GL_COMPILE_STATUS,
&bShaderCompiled);
•   }

```

- c. Once the shader objects have been loaded, a shader program is created. The program attaches and links the shader objects to the shader program.

It also provides the means for checking the compatibility of the shaders that are used to create a program (for instance, checking the compatibility between a vertex shader and a fragment shader). When they are no longer needed as part of a program object, the shader instances can be detached.